

Retrieval-Augmented Generation (RAG)

A Technical Overview of Architecture, Workflow, and Best Practices

Document Type: Technical Report
Prepared for: Engineering & AI Research Reference
Version 1.0

Table of Contents

1. Introduction
2. Why Retrieval-Augmented Generation?
3. Core Architecture
4. The RAG Workflow
5. Key Components
6. Types of RAG Architectures
7. Benefits and Limitations
8. Common Use Cases
9. Best Practices
10. Conclusion

1. Introduction

Retrieval-Augmented Generation (RAG) is an architectural pattern that combines large language models (LLMs) with an external knowledge retrieval system. Instead of relying solely on the static knowledge encoded during training, a RAG system retrieves relevant information from an external data source at query time and supplies it to the language model as additional context. This allows the model to generate answers that are more accurate, current, and grounded in verifiable information.

The term was introduced in a 2020 research paper by Facebook AI Research (now Meta AI), which proposed combining a pretrained retriever with a pretrained sequence-to-sequence generator to improve performance on knowledge-intensive natural language processing tasks. Since then, RAG has become one of the most widely adopted techniques for building reliable, enterprise-grade AI applications.

2. Why Retrieval-Augmented Generation?

Standard large language models have three well-known limitations that RAG is specifically designed to address:

- **Knowledge cutoff:** Models only know what existed in their training data up to a certain date and cannot access new or real-time information on their own.
- **Hallucination:** Without grounding, models may generate plausible-sounding but factually incorrect statements.
- **Lack of domain or private knowledge:** General-purpose models are not trained on an organization's internal documents, databases, or proprietary data.

By retrieving relevant, up-to-date, and domain-specific content before generating a response, RAG reduces hallucination, improves factual accuracy, and enables the model to answer questions about information it was never explicitly trained on — without the cost of retraining or fine-tuning the underlying model.

3. Core Architecture

At a high level, a RAG system consists of two cooperating subsystems: a **retriever**, which searches an external knowledge base for information relevant to a user's query, and a **generator**, which is a language model that produces a final response using both the original query and the retrieved content.

Component	Role	Typical Implementation
Data Ingestion	Loads and prepares source documents	Loaders, parsers, chunkers
Embedding Model	Converts text into numerical vectors	Sentence-transformer or API-based embeddings

Component	Role	Typical Implementation
Vector Store	Stores and indexes embeddings for search	FAISS, Pinecone, Weaviate, Chroma, Milvus
Retriever	Finds the most relevant chunks for a query	Dense, sparse, or hybrid search
Generator (LLM)	Produces the final answer using retrieved context	GPT, Claude, Llama, or similar models

4. The RAG Workflow

A typical RAG pipeline operates in two distinct phases: an offline **indexing phase** and an online **query phase**.

4.1 Indexing Phase (Offline)

- **Document loading:** Source documents (PDFs, web pages, databases, etc.) are collected.
- **Chunking:** Documents are split into smaller, semantically coherent passages to fit within context limits and improve retrieval precision.
- **Embedding:** Each chunk is converted into a dense vector representation using an embedding model.
- **Storage:** The vectors, along with references to their source text, are stored in a vector database for fast similarity search.

4.2 Query Phase (Online)

- **Query embedding:** The user's question is converted into a vector using the same embedding model.
- **Retrieval:** The system searches the vector database for the chunks most similar to the query vector.
- **Augmentation:** The retrieved chunks are inserted into a prompt template alongside the original question.
- **Generation:** The language model produces a final answer, grounded in the retrieved context.

5. Key Components

5.1 Chunking Strategy

The way documents are split significantly affects retrieval quality. Common strategies include fixed-size chunking, sentence- or paragraph-based chunking, and semantic chunking, which groups text based on meaning rather than length. Overlapping chunks are often used to preserve context across boundaries.

5.2 Embeddings

Embedding models map text into a high-dimensional vector space where semantically similar content is positioned closer together. The quality of the embedding model directly impacts how well the retriever can identify relevant passages.

5.3 Retrieval Methods

- **Dense retrieval:** Uses vector similarity (e.g., cosine similarity) to find semantically related content, even without exact keyword matches.
- **Sparse retrieval:** Traditional keyword-based methods such as BM25, effective for exact term matching.
- **Hybrid retrieval:** Combines dense and sparse methods, often improving overall accuracy.
- **Re-ranking:** A secondary model re-orders the initially retrieved results to surface the most relevant passages before they are sent to the generator.

6. Types of RAG Architectures

Variant	Description
Naive RAG	Basic retrieve-then-generate pipeline with a single retrieval pass.
Advanced RAG	Adds query rewriting, re-ranking, and filtering steps before generation to improve relevance.
Modular RAG	Composed of interchangeable modules (routing, fusion, memory) that can be reconfigured per use case.
Agentic RAG	An autonomous agent decides when, what, and how many times to retrieve, often iterating over multiple retrieval steps.
Graph RAG	Retrieves information from a knowledge graph instead of, or in addition to, a vector store, capturing relationships between entities.

7. Benefits and Limitations

7.1 Benefits

- Reduces hallucination by grounding responses in retrieved evidence.
- Enables access to current and domain-specific information without retraining the model.
- More cost-effective than fine-tuning for many knowledge-update use cases.
- Improves transparency, since retrieved sources can be cited alongside the generated answer.
- Easier to update: refreshing the knowledge base does not require retraining the model.

7.2 Limitations

- Retrieval quality directly limits answer quality — irrelevant retrieved content can mislead the generator.
- Adds system complexity: an additional indexing and retrieval pipeline must be built and maintained.
- Introduces latency, since retrieval happens before generation.
- Long or poorly chunked documents can still exceed context limits or dilute relevant information.

8. Common Use Cases

- **Enterprise knowledge assistants:** Answering employee questions using internal documentation, wikis, and policies.
- **Customer support:** Generating accurate responses grounded in product manuals and support tickets.
- **Research and legal analysis:** Summarizing and querying large collections of papers, contracts, or case files.

- **Code assistants:** Retrieving relevant code snippets or documentation to support software development.
- **Conversational search engines:** Combining web or document retrieval with natural-language answer generation.

9. Best Practices

- Choose a chunk size that balances context completeness with retrieval precision; test multiple sizes empirically.
- Use a high-quality, domain-appropriate embedding model, and re-evaluate it periodically.
- Incorporate a re-ranking step to filter out weakly relevant retrieved passages.
- Monitor retrieval quality separately from generation quality to isolate failure points.
- Provide citations or source references in generated answers to improve trust and verifiability.
- Regularly refresh the knowledge base to keep retrieved information current.

10. Conclusion

Retrieval-Augmented Generation bridges the gap between the static knowledge of large language models and the dynamic, ever-changing information organizations rely on. By pairing a retrieval system with a generative model, RAG produces responses that are more accurate, current, and traceable to their sources. As retrieval techniques, vector databases, and agentic reasoning continue to mature, RAG remains one of the foundational architectures for building trustworthy, production-grade AI systems.

— End of Document —